

Ferramentas para modelagem orientada a objetos

Introdução

Nesta aula sobre “Ferramentas para modelagem orientada a objetos”, exploraremos como essas ferramentas auxiliam na criação de sistemas organizados e eficientes. A modelagem orientada a objetos é essencial no desenvolvimento de software, pois nos permite visualizar e estruturar componentes de maneira clara e objetiva. Utilizaremos ferramentas como o Astah e o Android Studio para facilitar a criação de diagramas UML, que são fundamentais para representar classes, atributos e relações em sistemas complexos. Além disso, aprenderemos sobre a técnica dos cartões CRC, que nos ajuda a definir responsabilidades e colaboradores das classes de forma simples e intuitiva.

Instalação do Astah

Primeiramente, vamos iniciar com a instalação do Astah, que será nossa principal ferramenta de modelagem orientada a objetos, especificamente voltada para UML (Linguagem de Modelagem Unificada). Vamos acompanhar a instalação e a configuração inicial dessa ferramenta, além de uma visão geral de suas funcionalidades mais relevantes. O Astah é uma ferramenta desenvolvida pela empresa japonesa Change Vision, anteriormente conhecida como Gild, nome que ainda pode aparecer em alguns materiais mais antigos. O processo de instalação começa acessando o site oficial (asta.net), onde é possível fazer o download da versão gratuita para estudantes. Vale ressaltar que a licença estudantil tem duração de um ano e exige que você utilize um e-mail institucional para o cadastro.

Para iniciar, acesse o site e, na URL asta.net/products/free-students-license, preencha o formulário com seus dados, incluindo o nome da instituição de ensino e o e-mail institucional. Após o envio, você receberá um e-mail com a

chave de licença que deverá ser utilizada na ativação do software. Assim que o download for concluído, inicie a instalação do Astah. Não há necessidade de modificar as configurações padrão; basta seguir com a instalação clicando em “continuar” nas telas que surgirem. O Astah oferece versões para diversos sistemas operacionais, incluindo Windows, macOS e Linux. Após a instalação, a licença deve ser ativada acessando o menu “Help” na barra superior e selecionando a opção “License”. Em seguida, insira a chave recebida por e-mail.

O Astah UML é a versão que utilizaremos ao longo do curso, mas é importante mencionar que existem outras versões como o Astah Professional, que inclui funcionalidades extras, como diagramas DFD (Diagrama de Fluxo de Dados) e DER (Diagrama de Entidade-Relacionamento), que também podem ser úteis em alguns contextos de modelagem, embora não façam parte da UML. Para aqueles que possuem a licença do Astah Professional, terão acesso a todas essas funcionalidades adicionais.

Durante o uso da ferramenta, será necessário familiarizar-se com sua interface e recursos. O Astah permite a criação e manipulação de diversos tipos de diagramas UML, fundamentais para a modelagem orientada a objetos. Após a ativação da licença, você poderá começar a explorar esses diagramas, criar classes, definir associações, generalizações e outras relações importantes para o desenvolvimento de sistemas. A instalação do Astah, como vimos, é simples e rápida, permitindo que a ferramenta esteja pronta para uso em poucos minutos.

Visão geral do Astah

Nesta segunda parte da nossa aula, vamos explorar a visão geral do Astah, nossa ferramenta de modelagem UML. Ao abrir o Astah pela primeira vez, temos acesso a diversos tipos de diagramas, fundamentais para a modelagem orientada a objetos. Entre os principais diagramas UML disponíveis estão o diagrama de classe, de sequência, de casos de uso e de comunicação, abrangendo tanto diagramas estruturais (estáticos) quanto comportamentais (dinâmicos). Além disso, o Astah oferece alguns outros diagramas úteis, como DFD (Diagrama de Fluxo de Dados), DER (Diagrama de

Entidade-Relacionamento) e fluxogramas, que, apesar de não fazerem parte do padrão UML, podem ser aplicados em situações específicas.

Uma das funcionalidades que facilita o uso do Astah é o alinhamento automático. À medida que movemos os objetos no diagrama, o software cria guias que ajudam no alinhamento dos elementos, agilizando o processo de construção sem a necessidade de ajuste manual preciso. Isso torna o uso da ferramenta mais eficiente e esteticamente agradável, superando a dificuldade comum de posicionamento em diagramas. Outro destaque é a capacidade de gerar automaticamente diagramas de classe a partir de um código-fonte Java e, inversamente, gerar código Java a partir de diagramas, um recurso que pode ser estendido a outras linguagens, como C# e C++.

Ao iniciar o Astah, a tela principal exibe os menus com os principais diagramas disponíveis. Para adicionar objetos a um diagrama, basta clicar nos ícones correspondentes ou, de maneira mais ágil, dar um duplo clique na área de trabalho, o que gera automaticamente o elemento mais importante do diagrama. Por exemplo, ao criar um diagrama de classes, um duplo clique gera uma nova classe, e o mesmo procedimento pode ser aplicado a outros diagramas, como o de casos de uso ou de sequência, facilitando a inserção de atores, casos de uso e lifelines (linhas de vida).

O Astah também oferece recursos de personalização, permitindo ajustar aspectos visuais dos diagramas. Podemos, por exemplo, alterar a cor das classes ou pacotes para destacar similaridades ou diferenças. Além disso, o alinhamento de objetos pode ser feito automaticamente, com a distribuição simétrica dos elementos tanto na horizontal quanto na vertical, usando as ferramentas de auto layout. Isso garante que o diagrama mantenha um padrão visual organizado, mesmo quando os elementos são inseridos de maneira rápida.

Outra funcionalidade importante é a personalização dos objetos do diagrama. Ao clicar em uma classe, podemos renomeá-la diretamente na interface, além de configurar suas propriedades, como visibilidade, se a classe é abstrata ou não, e seus atributos e métodos. Esses detalhes podem ser usados posteriormente para gerar código-fonte a partir do diagrama. No menu “Tools”, podemos escolher a linguagem de programação e exportar o

diagrama para o diretório desejado. O Astah possibilita tanto a exportação de código quanto a importação de um código existente, oferecendo uma integração prática entre a modelagem e a implementação.

Instalação do Android Studio

Nesta terceira parte da nossa aula, vamos abordar a instalação do Android Studio, a IDE oficial do Google para o desenvolvimento de aplicativos Android, que também será utilizada em nosso curso. O processo de instalação começa acessando o site oficial do Android Studio, no endereço developer.android.com. Ao clicar no botão de download, será exibida a licença do software, que deve ser aceita para que o download seja iniciado. É importante destacar que o Android Studio é uma ferramenta gratuita e não requer nenhum tipo de registro pago. No entanto, ao usar uma conta Google, você terá acesso a recursos extras que podem facilitar o desenvolvimento.

O download e a instalação são bastante simples e seguem o padrão de qualquer programa. No site, a ferramenta detecta automaticamente o sistema operacional em uso (Windows, macOS ou Linux), oferecendo a versão adequada para cada plataforma. No caso do macOS, por exemplo, ele identifica se o dispositivo utiliza chip Intel ou o chip da Apple (M1, M2, etc.), enquanto no Windows, o instalador baixado será um arquivo .exe. Após o download, basta seguir com a instalação padrão, sem a necessidade de ajustes especiais, a menos que você seja um usuário mais avançado com necessidades específicas.

Uma das grandes vantagens do Android Studio é o suporte contínuo a atualizações, o que inclui novas funcionalidades, como a integração com inteligência artificial via chat, chamada Gemini. Esse recurso pode ajudar na busca por erros no código-fonte, sugerir melhorias e até auxiliar na escrita de novos códigos, funcionando como uma ferramenta complementar para o desenvolvedor. Além disso, a IDE oferece um sistema de comentários “to do”, que permite deixar tarefas organizadas diretamente no código-fonte, facilitando a gestão do desenvolvimento.

Outro destaque do Android Studio é o emulador Android, que permite simular diferentes modelos de dispositivos e versões do sistema operacional. Com essa funcionalidade, você pode testar o desempenho de seu aplicativo em dispositivos mais antigos ou recentes, ajustando a compatibilidade com diferentes APIs. Para aqueles que preferem testar diretamente em um dispositivo físico, o Android Studio oferece a opção de conectar um smartphone ou tablet Android ao computador, possibilitando a instalação e execução do aplicativo diretamente no hardware real, além de no emulador.

Visão geral do Android Studio

Nesta quarta parte da nossa aula, vamos aprofundar a visão geral do Android Studio, a IDE oficial do Google para o desenvolvimento de aplicativos Android, que será utilizada ao longo do curso. Ao abrir o Android Studio, encontramos a tela inicial, onde temos acesso ao código-fonte, ao emulador Android e ao log de erros, tudo organizado de forma a facilitar o desenvolvimento. Uma das principais vantagens de utilizar uma IDE como o Android Studio em comparação com editores de texto comuns é o suporte ao desenvolvimento inteligente. A IDE oferece funcionalidades como destaque de sintaxe, autocompletar de código e identificação de onde variáveis e métodos são utilizados, o que acelera o processo de desenvolvimento.

Além disso, o Android Studio suporta várias linguagens de programação, como Java, C, C++ e Kotlin, com a integração recente da inteligência artificial via chat, conhecida como Gemini. Essa ferramenta pode ser usada para tirar dúvidas, gerar código-fonte e sugerir melhorias no desenvolvimento. Embora não seja possível desenvolver um aplicativo completo apenas com sugestões automáticas, o Gemini pode auxiliar em tarefas mais simples ou em momentos de bloqueio criativo.

Ao abrir um projeto no Android Studio, você verá no lado esquerdo uma visão geral de todos os arquivos do projeto, como os arquivos de código-fonte Java. No lado direito, o emulador Android permite testar o aplicativo. A IDE também oferece um “Device Manager”, onde é possível gerenciar diferentes emuladores de dispositivos Android. Dependendo da capacidade do computador, pode-se abrir mais de um emulador ao mesmo tempo, mas para

configurações menos robustas, recomenda-se o uso de um emulador de cada vez. Um exemplo prático seria selecionar o modelo Pixel 4 com API versão 29 para testar a compatibilidade com versões mais antigas do Android, algo importante, visto que nem todos os usuários estão na última versão do sistema.

O Android Studio também permite criar novos dispositivos virtuais para testes. Além de smartphones, podemos desenvolver para tablets, relógios inteligentes (Wear OS), centrais multimídia para carros e até televisores, uma funcionalidade valiosa para quem deseja criar aplicativos para múltiplas plataformas Android. Outra possibilidade interessante é utilizar um dispositivo físico para testar o aplicativo. Basta conectar um smartphone Android ao computador e compilar o programa diretamente no dispositivo, sem depender do emulador.

Uma das funcionalidades mais úteis é a ferramenta de depuração, que permite identificar erros no código durante a execução. Adicionalmente, o Android Studio oferece o suporte ao controle de versão via Git e um terminal integrado para facilitar o gerenciamento de projetos. O Gemini também pode ser utilizado para automatizar partes do código, por exemplo, criando classes simples para operações matemáticas, como a soma de números, com explicações detalhadas. Embora a criação de um aplicativo completo dependa do desenvolvedor, esses recursos proporcionam suporte significativo ao processo.

Cartões CRC

Os cartões CRC (Class Responsibility Collaborator) são uma técnica utilizada para representar os componentes de um sistema, suas responsabilidades e as interações entre eles. CRC é a sigla para “Classe, Responsabilidade e Colaborador,” e essa técnica se mostra extremamente útil no design conceitual, pois proporciona uma visão de alto nível das classes, suas funções e como elas interagem entre si. O foco não é apenas identificar as classes, mas também compreender o que elas fazem e com quais outras classes colaboram. Originalmente, os cartões CRC foram criados como uma técnica de ensino, mas logo se tornaram uma prática comum na metodologia de

desenvolvimento ágil, especialmente na comunidade XP (Extreme Programming).

Cada cartão CRC é composto pelo nome da classe no topo e, logo abaixo, duas colunas: uma para listar as responsabilidades da classe e outra para indicar seus colaboradores. Um dos principais motivos de se utilizar cartões em papel é que eles facilitam a organização e evitam a criação excessiva de classes, que pode ocorrer em ferramentas digitais pela facilidade de manipulação. Ainda que existam softwares que simulem essa técnica, o uso de papel oferece um valor único para o raciocínio e a revisão colaborativa, tornando o processo mais dinâmico e intuitivo.

A técnica dos cartões CRC também auxilia na análise de coesão e acoplamento das classes. Se uma classe apresenta muitas responsabilidades, pode ser um indicativo de que ela não está seguindo o princípio da responsabilidade única, fundamental para garantir a coesão. Da mesma forma, se uma classe colabora com muitas outras, isso pode indicar um problema de acoplamento excessivo, dificultando a manutenção e a evolução do sistema. Portanto, o uso de cartões físicos permite que o grupo visualize e ajuste essas relações de maneira prática e rápida.

Após identificar as classes, responsabilidades e colaboradores, o próximo passo é espalhar os cartões em uma mesa e revisar suas interações. A troca de ideias e a colaboração em grupo são essenciais nessa fase. Por exemplo, imagine dois cartões: um representando a classe “Usuário” e outro a classe “Pedido”. No cartão da classe “Usuário”, as responsabilidades podem incluir armazenar e gerenciar informações do usuário, além de autenticar o acesso. A classe “Usuário” colabora com o banco de dados e a classe “Autenticador”, que será responsável pela autenticação propriamente dita. Já no cartão da classe “Pedido”, as responsabilidades podem ser criar um novo pedido e calcular seu total, colaborando com as classes “Produto” e “Usuário”.

Os benefícios de usar cartões CRC são variados. Além de proporcionar uma visualização simples e clara das responsabilidades das classes, essa técnica facilita a comunicação com stakeholders que não possuem um conhecimento técnico profundo. O uso dos cartões também promove o trabalho em equipe, pois permite que as ideias sejam discutidas e tratadas de maneira mais

eficiente. Ao focar nas responsabilidades e nas interações entre as classes, os cartões CRC ajudam a manter o design do sistema coeso e alinhado com os princípios de baixo acoplamento e alta coesão.

Ao longo desta aula, exploramos importantes ferramentas para modelagem orientada a objetos, como o Astah e o Android Studio, além da técnica dos cartões CRC. Vimos como essas ferramentas nos auxiliam na criação e organização de sistemas complexos, proporcionando um ambiente eficiente para a construção de diagramas UML e a definição clara de responsabilidades e interações entre as classes. Com essas ferramentas, o processo de design de software se torna mais visual e colaborativo, facilitando tanto o desenvolvimento quanto a comunicação entre os membros da equipe.

Conteúdo Bônus

Para se aprofundar no tema desta aula, sugiro que assista ao vídeo intitulado “Obtendo o Astah Gratuitamente: Guia Passo a Passo para Baixar o Software de Modelagem UML Sem Custos”, que está disponível no canal Maykon Dias no YouTube.

Referência Bibliográfica

ASCENCIO, A. F. G.; CAMPOS, E. A. V. de. **Fundamentos da programação de computadores**. 3.ed. Pearson: 2012.

BRAGA, P. H. **Teste de software**. Pearson: 2016.

GALLOTTI, G. M. A. (Org.). **Arquitetura de software**. Pearson: 2017.

GALLOTTI, G. M. A. **Qualidade de software**. Pearson: 2015.

MEDEIROS, E. **Desenvolvendo software com UML 2.0 definitivo**. Pearson: 2004.

MORAIS, I. S. de. (Org.). **Engenharia de software**. Pearson: 2017.

PFLEEGER, S. L. **Engenharia de software: teoria e prática.** 2.ed. Pearson: 2003.

SOMMERVILLE, I. **Engenharia de software.** 10.ed. Pearson: 2019.